

RESTAURARTE

DOCUMENTAÇÃO DO PROJETO

Trabalho A3 – Algoritmos e Programação

Ian Fernandes Guimarães
Cibely Rodrigues de Pontes
Daniel Camargo da Cunha
Miguel Dolabella Miranda
Pedro Henrique Penha Piantino
Vitor Ribeiro Dalmazio Machado

Descrição do Projeto

Tema escolhido

RestaurARTE é um sistema de gerenciamento de restaurante executado via terminal, desenvolvido em Java com persistência em banco de dados SQLite. O sistema simula o ciclo completo de atendimento: cadastro e login de cliente, consulta ao cardápio, montagem de pedido, edição e exclusão de itens, encerramento da conta e coleta de avaliações.

Problema que o sistema resolve

Pequenos restaurantes frequentemente controlam pedidos de forma manual ou não possuem nenhum sistema de histórico de clientes. O RestaurARTE resolve isso oferecendo: acesso simplificado sem senha apenas pelo nome, histórico digital de cada cliente com seus pedidos, controle de pedidos em aberto e encerrados, possibilidade de editar o pedido antes de finalizar, e coleta estruturada de avaliações com nota e comentário.

Público-alvo

Funcionários do restaurante e Clientes do restaurante, já que interagem diretamente com o sistema para montar seus pedidos e avaliar a experiência.

Diagrama ou Estrutura do Sistema

Descrição das classes

- `Main.java` - Inicializador do programa
- `TUI.java` - Gerenciamento gráfico do programa utilizando o terminal como tela
- `Utils.java` - Úteis que compoem o sistema através de funções que são utilizadas recursivamente por todas as classes
- `databaseService.java` - Responsável pelo gerenciamento do cliente
- `clienteController` / `clienteService` - Responsável pelo gerenciamento do cliente
- `pratosController` / `pratosService` - Responsável pelo gerenciamento dos pratos
- `pedidoController` / `pedidoService` - Responsável pelo gerenciamento dos pedidos
- `avaliacaoController` / `avaliacaoService` - Responsável pelo gerenciamento das avaliações

Responsabilidades de cada classe

- **TUI:** exibe menus, lê entradas do usuário e mostra estado da sessão do cliente e pedido atual
- **Services:** executam toda a lógica de database através das queries SQL via JDBC
- **Controllers:** Gerenciam as informações de cada classe como: id, nome, preço e datas
- **databaseService:** garante uma única conexão ativa e cria as tabelas automaticamente na primeira execução.

Fluxo básico do sistema

Inicialização do programa -> Menu Principal -> Login/Cadastro -> Exibição do Perfil do Usuário -> Menu Restaurante, com opções de Ver Cardápio, Ver Pedido, Encerrar e Pagar, e Sair -> Ver Cardápio, permite adicionar pratos -> Ver Pedido, permite visualizar, editar e excluir itens -> Encerrar e Pagar, registra o pagamento -> Avaliação solicitada ao final -> Retorno ao Menu Principal.

Levantamento de Requisitos

Requisitos Funcionais

- **Cadastro:** o sistema deve registrar automaticamente um novo cliente quando o nome digitado não existir no banco, armazenando nome e data de entrada.
- **Listagem:** o sistema deve listar o cardápio completo: ID, nome, preço, tempo de preparo, os itens do pedido atual com total calculado, o histórico de pedidos do cliente e todas as avaliações registradas com a média geral.
- **Edição:** o sistema deve permitir adicionar pratos ao pedido em aberto a qualquer momento antes do encerramento.
- **Exclusão:** o sistema deve permitir remover um prato específico do pedido em aberto, informando o ID do item a ser excluído.
- **Encerramento do sistema:** o sistema deve permitir encerrar e fazer pagamento do pedido, assim encerrando o programa de forma controlada pela opção de sair, no menu principal.

Requisitos Não Funcionais

- **Linguagem Java:** o sistema é desenvolvido integralmente em Java, utilizando os recursos da linguagem.
- **Execução via terminal:** toda a interação ocorre exclusivamente em ambiente de linha de comando (TUI), sem interface gráfica.
- **Código organizado:** o projeto segue arquitetura em camadas: TUI, Services, Controllers, Database com responsabilidades bem delimitadas entre as classes.
- **Uso de Git/GitHub:** o código é versionado com Git e hospedado no GitHub, com histórico de commits e uso de duas branches: main e teste para controle de desenvolvimento.